Avaliação Técnica

Desenvolvedor Front End — Angular

Avalia conhecimentos práticos em Angular 17+, RxJS, NgRx/Signals, Angular Material, testes unitários, TypeScript e integração com APIs REST.

⏱ **Prazo de entrega**	📦 **Entrega**
4 dias corridos após recebimento	Encaminhar este documento com as perguntas respondidas e com o link do código público em sua conta do GitHub

1. TypeScript e Qualidade de Código

1.1. Refatoração

Considerando seus conhecimentos de TypeScript, qualidade de código e boas práticas, quais melhorias você faria no seguinte código:

```
class Produto {
 id: any;
 descricao: any;
 quantidadeEstoque: any;

 constructor(id: any, descricao: any, quantidadeEstoque: any) {
   this.id = id;
   this.descricao = descricao;
   this.quantidadeEstoque = quantidadeEstoque;
 }
}

class Verdureira {
 produtos: any;

 constructor() {
   this.produtos = [
     new Produto(1, 'Maçã', 20),
     new Produto(2, 'Laranja', 0),
     new Produto(3, 'Limão', 20)
   ];
 }

 getDescricaoProduto(produtoId: any) {
   let produto;

   for (let index = 0; index < this.produtos.length; index++) {
     if (this.produtos[index].id == produtoId) {
       produto = this.produtos[index];
     }
   }

   return produto.id + ' - ' + produto.descricao + ' (' + produto.quantidadeEstoque + 'x)';
 }

 hasEstoqueProduto(produtoId: any) {
   let produto;

   for (let index = 0; index < this.produtos.length; index++) {
     if (this.produtos[index].id == produtoId) {
       produto = this.produtos[index];
     }
   }

   if (produto.quantidadeEstoque > 0) {
     return true;
   } else {
     return false;
   }
 }

}
```

Resposta:

```
class Produto {
 id: number;
 descricao: string;
 quantidadeEstoque: number;

 constructor(id: number, descricao: string, quantidadeEstoque: number) {
   this.id = id;
   this.descricao = descricao;
   this.quantidadeEstoque = quantidadeEstoque;
 }
}

class Verdureira {
 produtos: Produtos[];

 constructor() {
   this.produtos = [
     new Produto(1, 'Maçã', 20),
     new Produto(2, 'Laranja', 0),
     new Produto(3, 'Limão', 20)
   ];
 }

 private getProdutoById(produtoId:number):Produto | undefined {
  return this.produtos.find((p)=> p.id === produtoId);
 }

 getDescricaoProduto(produtoId: number): string  {
   const produto = this.getProdutoById(produtoId)
   if (!produto){
      return 'Produto não encontrado';
   }

   return `${produto.id}-${produto.descricao} (${produto.quantidadeEstoque}x)`;
 }

 hasEstoqueProduto(produtoId: number): boolean {
  const produto = this.getProdutoById(produtoId);

  return !!produto && produto.quantidadeEstoque >
 }}
```

Explicação:

Garantindo tipagem correta esperada das props dos objetos e funções assim como o tipo de dado esperado nos returns.

Foi criada uma função para pegar o id do produto, tirando a necessidade de for para pesquisa e também utilizando a função para verificar se o produto existe.

Criados returns que reflitam caso o produto seja undefined.

1.2. Generics e tipos utilitários

Implemente uma função genérica filtrarEPaginar<T> que recebe um array, um predicado de filtro e parâmetros de paginação (página e tamanho). A função deve retornar os itens da página atual e o total de registros filtrados. Use tipagem completa — sem any.
Exemplo de parâmetros:

```
filtrarEPaginar<T>(
 data: T[],
 filterFn: (item: T) => boolean,
 params: PaginaParams
): Pagina<T>
```

Demonstre o uso com um exemplo concreto (ex: array de usuários com filtro por nome).

Resposta:

- Código na branch answer1.2 com implementação real e arquivos.
- Models possui as estruturas dos objetos filtrados.
- Functions possui a função filtrarEPaginar.
- App possui o exemplo com mock de usuários e mostrando a implementação.

Fiz pagina 1-based e utilizei normalização para evitar página 0 e tamanho 0.

2. Angular — Fundamentos e Reatividade

2.1. Change Detection e OnPush

O componente abaixo usa ChangeDetectionStrategy.OnPush, mas o nome não é exibido na tela. Identifique o problema, explique o motivo e proponha a correção — sem alterar a estratégia, sem modificar PessoaService e sem remover o setInterval.

```
import { ChangeDetectionStrategy, Component, Injectable, OnInit, OnDestroy } from '@angular/core';
import { of, Subscription } from 'rxjs';
import { delay } from 'rxjs/operators';

@Injectable()
class PessoaService {
 /** @description Mock de uma busca em API com retorno em 0.5 segundos */
 buscarPorId(id: number) {
   return of({ id, nome: 'João' }).pipe(delay(500));
 }
}

@Component({
 selector: 'app-root',
 providers: [PessoaService],
 changeDetection: ChangeDetectionStrategy.OnPush,
 template: `<h1>{{ texto }}</h1>`,
})
export class AppComponent implements OnInit, OnDestroy {
 texto: string;
 contador = 0;
 subscriptionBuscarPessoa: Subscription;
 constructor(private readonly pessoaService: PessoaService) {}

 ngOnInit(): void {
   this.subscriptionBuscarPessoa = this.pessoaService.buscarPorId(1).subscribe((pessoa) => {
     this.texto = `Nome: ${pessoa.nome}`;
   });
```

```
  setInterval(() => this.contador++, 1000);
 }

 ngOnDestroy(): void {
   /** ... */
 }
}
```

Resposta:

O problema é que ChangeDetectionStrategy.onPush não roda automaticamente em qualquer mudança, o que está acontecendo nesse cenário é que:

O subscribe roda assincronamente e a mudança do texto não dispara change detection pois não teve mudança no @Input e nem interação do usuário.

Nesse caso a melhor solução seria forçar a detecção manualmente, pois isso sinalizaria uma mudança relevante fora dos gatilhos padrão. Em código faria algo assim:

```
import { ChangeDetectionStrategy, Component, Injectable, OnInit, OnDestroy, ChangeDetectorRef } from
'@angular/core';
import { of, Subscription } from 'rxjs';
import { delay } from 'rxjs/operators';

@Injectable()
class PessoaService {
 /** @description Mock de uma busca em API com retorno em 0.5 segundos */
 buscarPorId(id: number) {
   return of({ id, nome: 'João' }).pipe(delay(500));
 }
}

@Component({
 selector: 'app-root',
 providers: [PessoaService],
 changeDetection: ChangeDetectionStrategy.OnPush,
 template: `<h1>{{ texto }}</h1>`,
})
export class AppComponent implements OnInit, OnDestroy {
 texto: string;
 contador = 0;
 subscriptionBuscarPessoa: Subscription;
 constructor(
         private readonly pessoaService: PessoaService,
         private readonly cdr: ChangeDetectorRef
 ) {}

 ngOnInit(): void {
   this.subscriptionBuscarPessoa = this.pessoaService.buscarPorId(1).subscribe((pessoa) => {
     this.texto = `Nome: ${pessoa.nome}`;
     this.cdr.markForCheck();
   });

   setInterval(() => this.contador++, 1000);
 }

 ngOnDestroy(): void {
   /** ... */
 }
}
```

Com markForCheck(), temos precisão na atualização, sem risco de over-render, é compatível com OnPush, agenda para o próximo ciclo, evita erros e é uma solução escalável para múltiplos fluxos async.

2.2. RxJS — eliminando subscriptions aninhadas

Refatore o código abaixo eliminando o subscribe dentro de subscribe. Use operadores RxJS adequados, evite memory leaks e explique brevemente sua escolha de operador:

```
ngOnInit(): void {
  const pessoaId = 1;

  this.pessoaService.buscarPorId(pessoaId).subscribe(pessoa => {
    this.pessoaService.buscarQuantidadeFamiliares(pessoaId).subscribe(qtd => {
      this.texto = `Nome: ${pessoa.nome} | familiares: ${qtd}`;
    });
  });
}
```

Resposta:

```
import { switchMap, map } from 'rxjs/operators';

ngOnInit(): void {
  const pessoaId = 1;

  this.pessoaService.buscarPorId(pessoaId).pipe(
    switchMap(pessoa) =>
      this.pessoaService.buscarQuantidadeFamiliares(pessoaId).pipe(
       map((qtd) => `Nome: ${pessoa.nome} | familiares: ${qtd}`)
      )
     ),
     takeUntilDestroyed()
   ).subscribe({ pessoa, qtd }) => {
      this.texto = `Nome: ${pessoa.nome} | familiares: ${qtd}`;
   });
}
```

Removido o subscribe aninhado

Substituído por um fluxo RxJS com pipe, tornando o código mais legível e reativo.

Uso de switchMap

Permite encadear chamadas assíncronas dependentes (buscar pessoa → buscar familiares), cancelando execuções anteriores se necessário.

Uso de map

Responsável por transformar o resultado final no formato desejado (texto), mantendo a lógica fora do subscribe.

Prevenção de memory leak

Adicionado takeUntilDestroyed() para gerenciar automaticamente o ciclo de vida da inscrição.

2.3. RxJS — busca com debounce

Implemente um campo de busca reativo em um componente Angular que:

- Aguarde 500 ms após o usuário parar de digitar antes de disparar a requisição (debounce)
- Cancele a requisição anterior caso o usuário digite novamente (evite race condition)
- Exiba um indicador de loading enquanto a requisição está em andamento
- Gerencie a subscription sem memory leak

Mostre o serviço, o componente e o template com async pipe.

Resposta:

- Código na branch answer2.3 com implementação real e arquivos.
- Models possui as estruturas das interfaces criadas.
- Services possui a implementação do serviço
- App possui a implementação do campo e o template html está no app.html

2.4. Performance — OnPush e trackBy

Considere uma lista com centenas de itens renderizados com @for (ngFor). Explique:

- Por que usar trackBy melhora a performance e como implementá-lo corretamente
- Como ChangeDetectionStrategy.OnPush pode reduzir ciclos desnecessários de detecção neste cenário
- Qual seria o impacto de usar a estratégia Default neste caso

Resposta:

Quando não usamos trackBy, o Angular usa a identidade por referência, logo se a lista for recriada, ele destrói todos os DOM nodes e recria tudo do zero. Com trackBy, estamos especificando que aquele é o mesmo item se tiver o mesmo ID. atualizando somente itens que mudaram e evita re-render completo. A implementação seria algo assim:

template

```
<li *ngFor="let item of itens; trackBy: trackById">
        {{ item.nome }}
</li>
```

component

```
trackById(index: number, item: Item): number {
        return item.id;
}
```

A estratégia de OnPush pode ser utilizada também pois só roda Change Detection quando @Input muda, ocorre um evento na view, observable com async ou definindo manual com markForCheck por exemplo. Com isso, a lista não muda a menos que algo relevante aconteça e isso acaba com os ciclos desnecessários de renderização.

Com estratégia Default, teremos problemas pois executa detecção global constante o que degrada a performance com listas grandes, gera renderizações desnecessárias e gera um alto uso de CPU.

3. Gerenciamento de Estado

3.1. Angular Signals — estado local

Crie um componente de contador de itens no carrinho usando exclusivamente Signals. O componente deve expor:

- Um signal para a lista de itens
- Um computed para o total (quantidade × preço)
- Um método para adicionar e remover itens
- Um output() que emite sempre que o total mudar

Resposta:

- Código na branch answer3.1 com implementação real e arquivos.

- Models possui a estrutura da interface do cartItem.
- App possui a implementação do componente com Signals e o template html está no app.html.

3.2. Gerenciamento de Estado com NgRx (Feature To-do)

Implemente a estrutura de estado para uma lista de tarefas (To-do) utilizando os padrões recomendados do NgRx. A implementação deve incluir:

- **Actions:** Definir ações para: `loadTodos`, `loadTodosSuccess`, `loadTodosError` e `toggleTodoComplete`.
- **Reducer:** Implementar o estado inicial e a transição de estados usando `createReducer`, garantindo a tipagem forte do estado.
- **Selectors:** Criar seletores utilizando `createSelector`:
  - `selectAllTodos`: Retorna a lista completa.
  - `selectPendingTodos`: Retorna apenas as tarefas não concluídas.
- **Effect:** Criar um `Effect` que gerencie o fluxo assíncrono: ao disparar `loadTodos`, deve realizar uma chamada HTTP (mockada) e despachar a ação de sucesso ou erro conforme o resultado.

Não é necessário implementar o back-end — use HttpClient com uma URL fictícia.

Resposta:

- Código na branch answer3.2 com estrutura de arquivos.
- Models possui uma estrutura base do model para Todo.
- Store possui toda a estrutura de estado com NgRx.
- Service possui um back-end fake com HttpClient e URL fictícia.

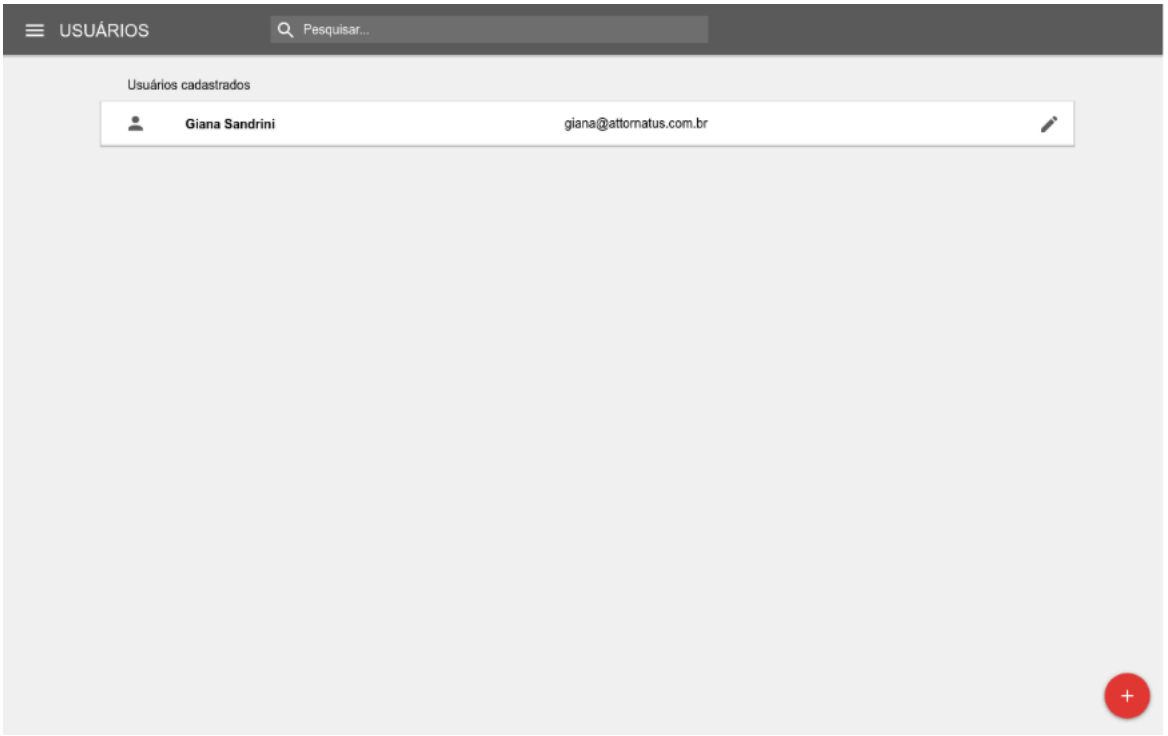
4. Desafio Prático — Aplicação Angular

Stack obrigatória: Angular 17+ · Angular Material · NgRx ou Signals · RxJS · Vitest ou Jest

4.1 O que construir

Reproduzir o protótipo de uma listagem de usuários. A listagem deverá ter as seguintes funcionalidades:

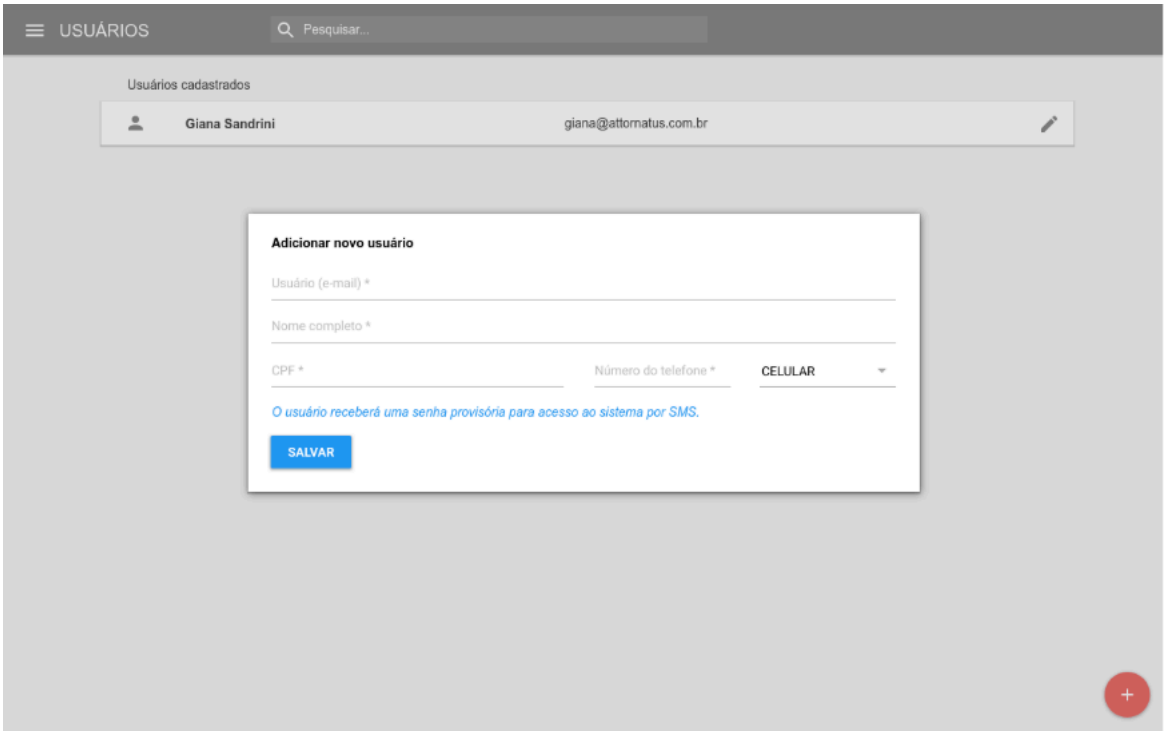
Tela de listagem de usuários:



Listagem de usuários:

- Cards com os campos: nome, e-mail e botão de editar
- Filtro por nome com debounce de 300ms
- Estado de loading durante o carregamento e mensagem de erro em caso de falha
- Os dados podem vir de uma API mockada (JSON Server, MSW ou array estático em serviço)

Modal de cadastro de novo usuário com abertura através do botão vermelho que aparece na listagem:



Criação e edição de usuário (em modal):

- Formulário reativo com campos: e-mail (obrigatório), nome (obrigatório), cpf (obrigatório), telefone (obrigatório) e tipo de telefone.
- Validação com mensagens de erro por campo
- Botão de salvar desabilitado enquanto o formulário estiver inválido
- Quando for edição o formulário deve ser preenchido automaticamente

4.2 Requisitos técnicos

- Pelo menos 2 operadores RxJS além de map e tap em uso real (ex: switchMap, forkJoin, catchError, debounceTime)
- Componentes standalone
- Subscriptions gerenciadas sem memory leaks (takeUntilDestroyed, take, async pipe ou unsubscribe manual no ngOnDestroy)
- Cobertura de testes acima de 60% (Preferencialmente usando Vitest ou Jest)

4.3 Diferenciais (não obrigatórios)

- Nx Monorepo com separação em bibliotecas (ex: feature-users, data-access-users, ui)
- Paginação na listagem
- Validação de formato dos dados do formulário (e-mail,cpf e telefone)
- Melhorias no projeto em relação a tela apresentada no protótipo

---

⚠ Inclua um README.md com instruções de instalação e execução. Projetos que não rodam localmente não serão avaliados.

✅ Dúvidas? Entre em contato pelo e-mail informado no processo seletivo. Boa sorte! 🚀

Estamos ansiosos para conhecer a sua solução.